

קבוצות זרות – Union Find

הגדרת המבנה:

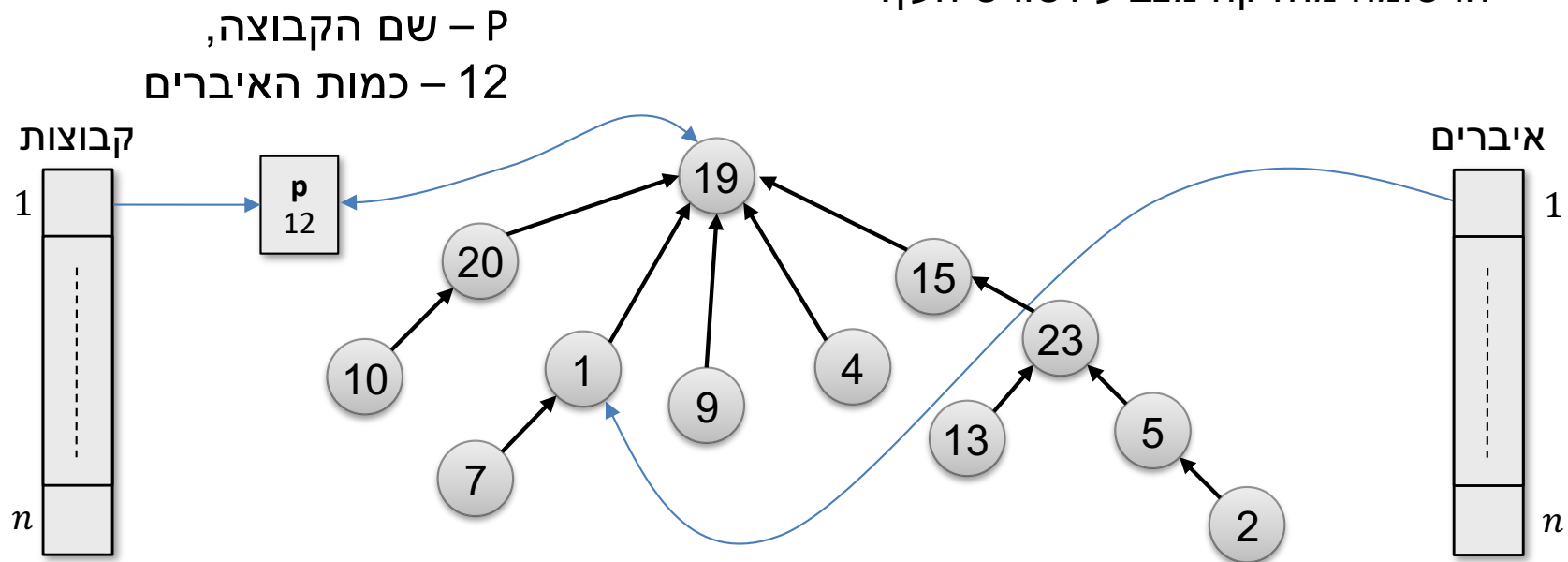
נתונים n איברים, אשר יסומנו $1, 2, \dots, n$.
איברים אלו מחולקים לאורך כל ריצת המבנה לקבוצות זרות.
בתחילת ריצת המבנה, כל האיברים מחולקים לקבוצות בגודל 1 (Singleton).

המבנה תומך בשתי הפעולות הבאות:

- **Find(i)** החזר את שם הקבוצה אליה שייך האיבר i . (לא האיבר עצמו, אלא הקבוצה)
- **Union(p, q)** אחד את שתי הקבוצות המזוהות ע"י p ו- q - והחזר את שם הקבוצה החדשה.
לאחר פעולה זו, הקבוצות p ו- q - נהרסות ובמקומן נבנית הקבוצה $p \cup q$.
נשים לב ש- q זה בעצם הקבוצות.

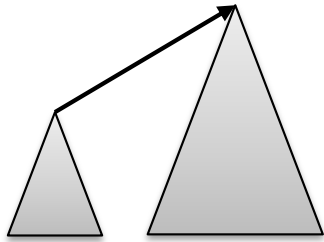
מימוש Union Find בעזרת עצים הפוכים

- לכל קבוצה ניצור עץ הפוך (עץ שבו הבנים מצביעים לאבותיהם) מכל איברי הקבוצה.
- שורש כל עץ הפוך יצביע על רשומה שתכיל את שם הקבוצה ומספר איבריה.
- בנוסף נחזיק שני מערכים:
 1. מערך איברים: ממפה בין איבר לצומת המתאים לו בעץ הפוך.
 2. מערך קבוצות: ממפה בין מספר הקבוצה לרשומה המתאימה לקבוצה.הרשומה מחזיקה מצביע לשורש העץ.

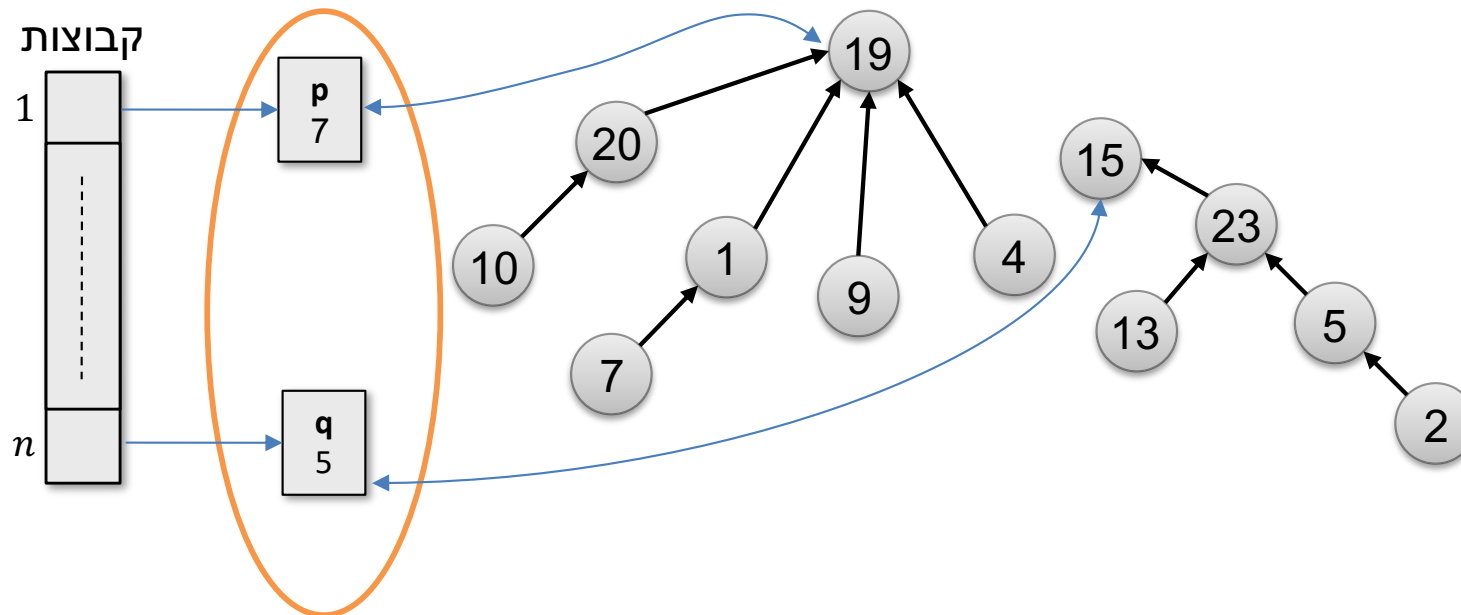


מימוש פעולת Union

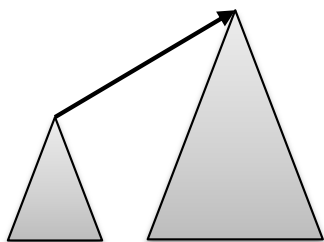
- ממערך הקבוצות נגש לשורשי שני העצים.
- נפנה את שורש קבוצה אחת להצביע על שורש הקבוצה השנייה.
- נעדכן את גודל הקבוצה החדשה ונסלק את כותרת הקבוצה השנייה ממערך הקבוצות.
- סיבוכיות זמן: $O(1)$ במקרה הגרוע.



לבצע איחוד בין אלו

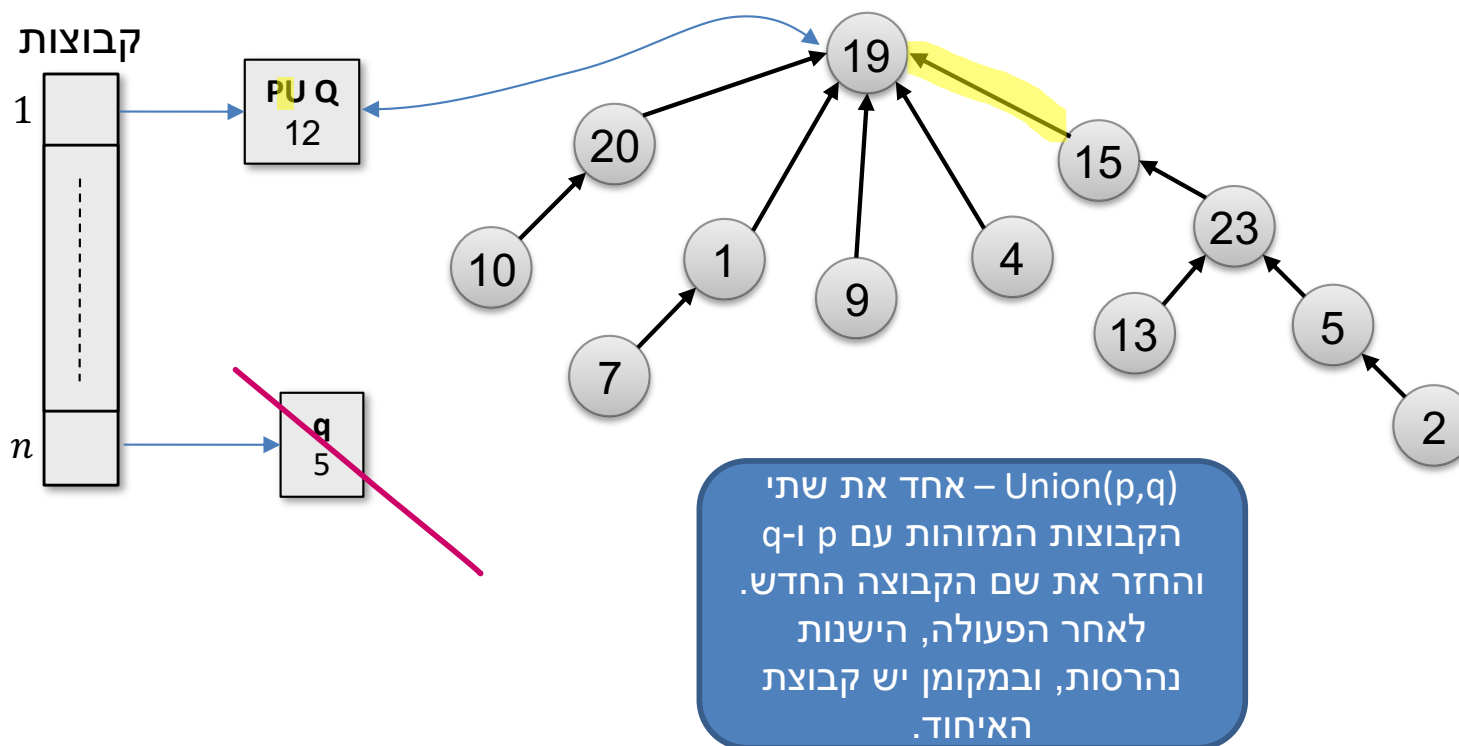


מימוש פעולת Union



- ממערך הקבוצות נגש לשורשי שני העצים.
- נפנה את שורש קבוצה אחת להצביע על שורש הקבוצה השנייה.
- נעדכן את גודל הקבוצה החדשה ונסלק את כותרת הקבוצה השנייה ממערך הקבוצות.
- סיבוכיות זמן: $O(1)$ במקרה הגרוע. - רק שיחקנו עם מצביעים.

לבצע איחוד בין אלו



מימוש פעולת Find

- ממערך האיברים נגש לצומת בעץ ההפוך.
- נטפס בעץ ההפוך עד שנגיע לשורש. ממנו נגש למתאר הקבוצה ונחזיר את שמה של הקבוצה.

סיבוכיות זמן: $O(n)$ במקרה הגרוע.

ללא איחוד לפי גודל, יתכן ואורך מסלול החיפוש יהיה n .
נראה כיצד איחוד לפי גודל משפר זאת.



דוגמא לאיך נוצר שרוך ולכן זה $O(n)$ במקרה הגרוע.
נניח וקיבלו את האיברים האלו: 10, 20, 18
איחוד בין 10 ו-20 נקבל

10 → 20

10 → 20 → 18

כעת מבקשים איחוד בין הקבוצה של 18 לבין הקבוצה של 10 ו-20 ונקבל:
במקרה הזה, אם היינו עושים איחוד לפי גודל, אז 18 היה מתחבר ל-20, אבל הפעם 20 יתחבר ל-18. אבל בגלל שאין איחוד לפי גודל אז זה יכול לקרות.

שיפור 1 – איחוד קבוצות לפי גודל

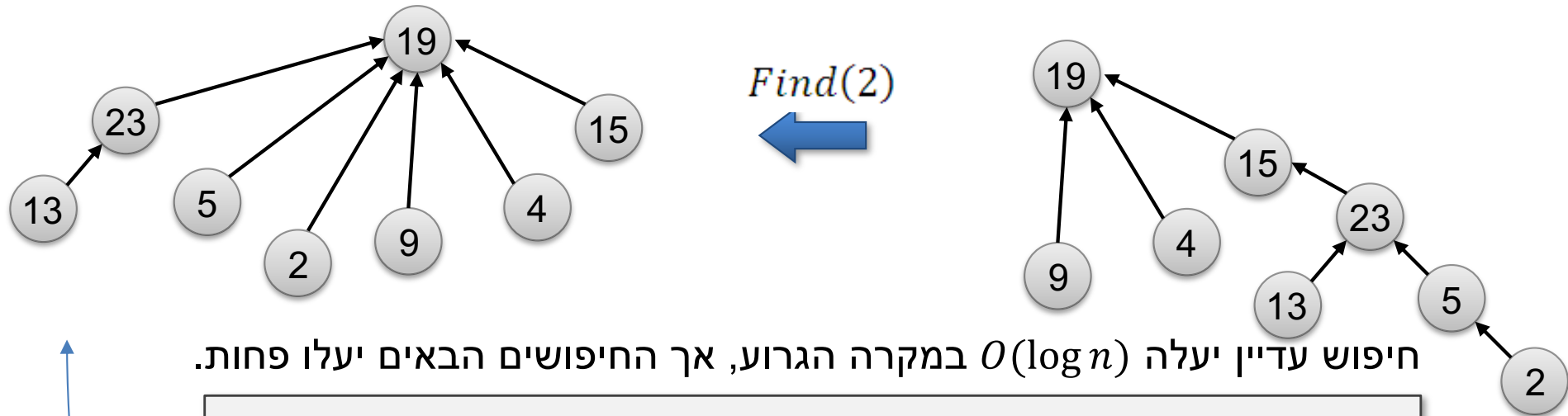
אם נבצע איחוד לפי גודל, כל פעולת Find תיקח $O(\log n)$ במקרה הגרוע.

הוכחה: לכל איבר i במבנה:

- המרחק ההתחלתי של i משורש הקבוצה הוא 0, כיוון ש- i האיבר היחיד בקבוצה.
 - בכל פעם שהמרחק של i מהשורש גדל ב-1, גודל הקבוצה בה i נמצא גדל פי 2 לפחות. למה?
- אם השורש לא השתנה, זה אומר שהוא היה בקבוצה הקטנה. המרחק של השורש גדל ב-1 כי מחברים את השורש של הקבוצה הקטנה לשורש של הקבוצה הגדולה. גודל הקבוצה הגדולה הוא לפחות בגודל של הקבוצה הקטנה, לכן זה לפחות פי 2.
- גודל הקבוצה המקסימלי הוא n ועל כן המרחק בין כל צומת לשורש לא יכול לעלות על $\log_2 n$. למה?
- גודל הקבוצה המקסימלי הוא n . אנחנו מגדילים את המרחק מצומת לשורש רק במידה והצומת נמצא בקבוצה הקטנה מבין ה-2 בעת האיחוד. בכל פעם כזאת, גודל הקבוצה שלו גדל לפחות פי 2. אז לא ייתכן שצומת כלשהו נמצא בקבוצה הקטנה יותר מ- $\log n$ פעמים. לכן כמות הפעמים שהוספנו קשת לשורש החדש, לא יכולה להיות גדולה מ- $\log n$. (לא נתעכב על ההוכחה).

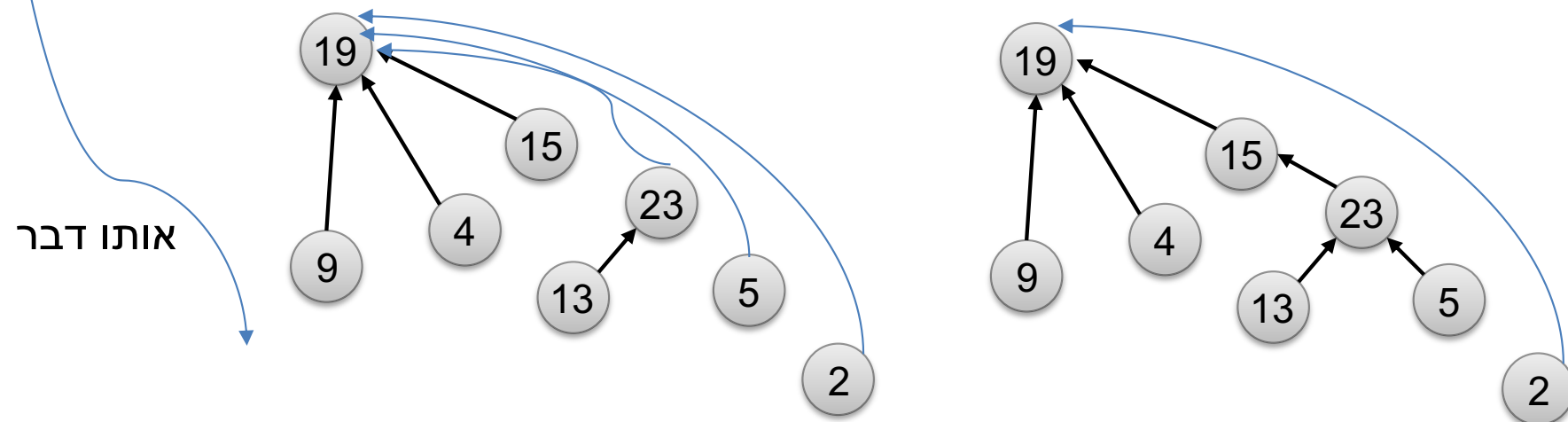
שיפור 2 – כיווץ מסלולים

לאחר מציאת האיבר ה- i , נסרוק שוב את המסלול בין i לשורש ונחבר כל צומת במסלול לשורש.



חיפוש עדיין יעלה $O(\log n)$ במקרה הגרוע, אך החיפושים הבאים יעלו פחות.

משפט: אם נשתמש באיחוד לפי גודל ובכיווץ מסלולים, סיבוכיות הזמן של m פעולות Union, Find הינה $O(m \log^* n)$ (בהרצאות).
לכן, סיבוכיות הזמן המשוערכת של שתי הפעולות יחד הינה $O(\log^* n)$.



שיפור 2 – מה זה $\log^* n$?

$$\log^* n = \frac{i}{\min\{i | \log(\log(\log(\log(\dots \log(n)) \dots)) \leq 1\}}$$

In computer science, the iterated logarithm of n , written $\log^* n$ (usually read "log star"), is the number of times the logarithm function must be iteratively applied before the result is less than or equal to 1.

Example:

1) $\log^* (n)=5$ where n = Number of atom in universe

בקורס זה לא $O(1)$

תרגיל 1 – קבוצות עם מקסימום

בנוסף לפעולות Union ו-Find, נרצה להוסיף פעולה $\text{Max}(i)$ אשר תחזיר את האיבר המקסימלי בקבוצה בה נמצא i .

פתרון

נשתמש במבנה הנתונים Union-Find במימוש המתקדם ביותר שראינו (כיווץ מסלולים ואיחוד לפי גודל), כך ששורש כל עץ הפוך יחזיק בשדה נוסף max את האיבר המקסימלי בקבוצתו.

$\text{Find}(i)$ כמו קודם.

$\text{Max}(i)$ נבצע $\text{Find}(i)$ ונחזיר את ערך השדה max שבשורש.

$\text{Union}(p,q)$ נבצע Union כרגיל ונעדכן את שדה max בשורש העץ החדש להיות הערך המקסימלי מבין שני ערכי שדות ה- max של הקבוצות p,q .

סיבוכיות הזמן המשוערכת של שלוש הפעולות ביחד הינה $O(\log^* n)$.

תרגיל 1 – קבוצות עם מקסימום

בנוסף לפעולות Union ו-Find, נרצה להוסיף פעולה $\text{Max}(i)$ אשר תחזיר את האיבר המקסימלי בקבוצה בה נמצא i .

פתרון –

נשתמש במבנה UF (עם כיווץ מסלולים ואיחוד לפי גודל), כך שמתאר כל קבוצה יחזיק גם שדה max שהוא האיבר המקסימלי בקבוצה.

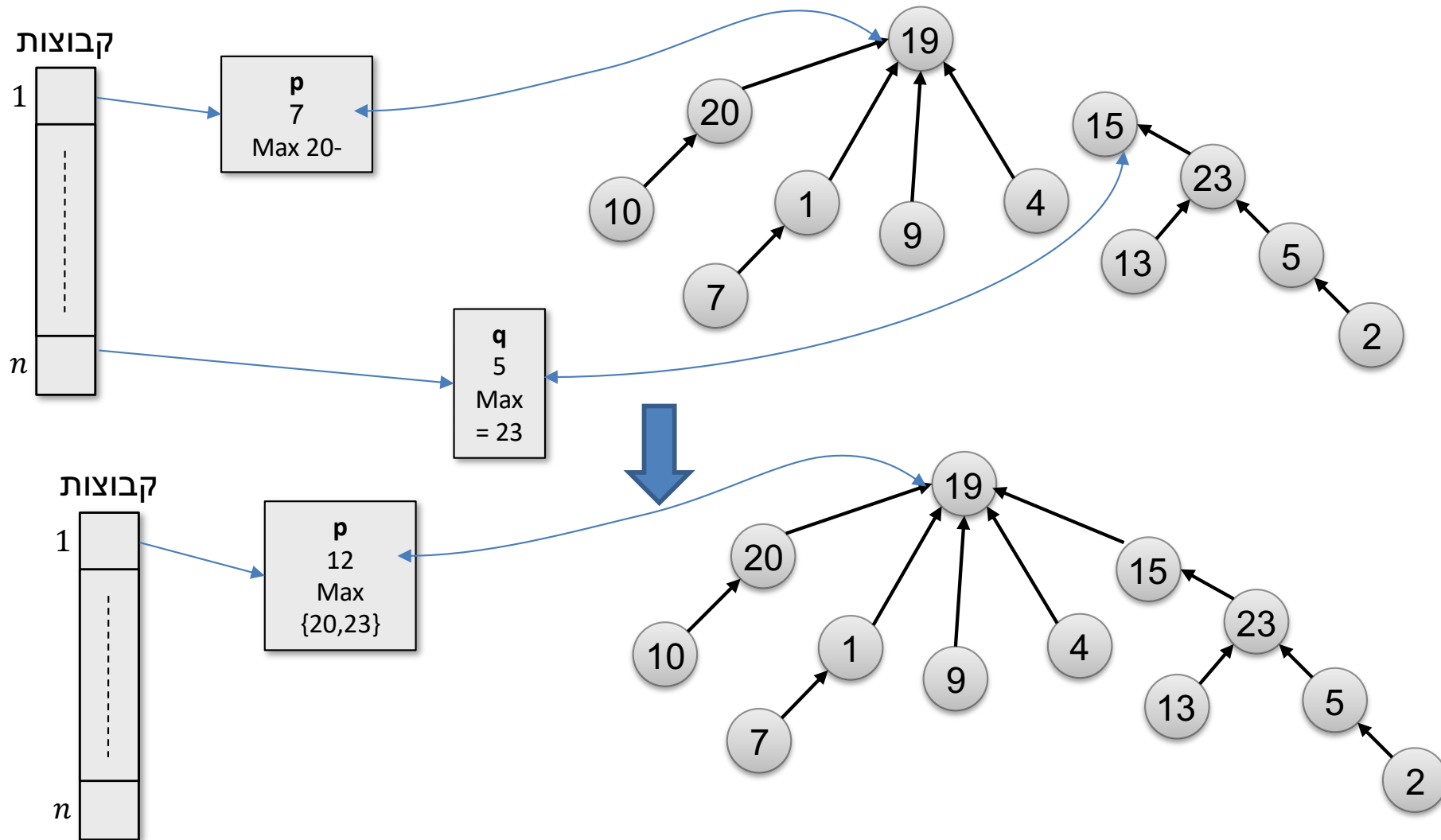
Find – נשאר אותו דבר.

Union – יישאר כרגיל רק נשווה וניקח את המקסימלי מבין שני המקסימום בקבוצות המקוריות.

תרגיל 1 – קבוצות עם מקסימום

בנוסף לפעולות Union ו-Find, נרצה להוסיף פעולה $\text{Max}(i)$ אשר תחזיר את האיבר המקסימלי בקבוצה בה נמצא i .

כל

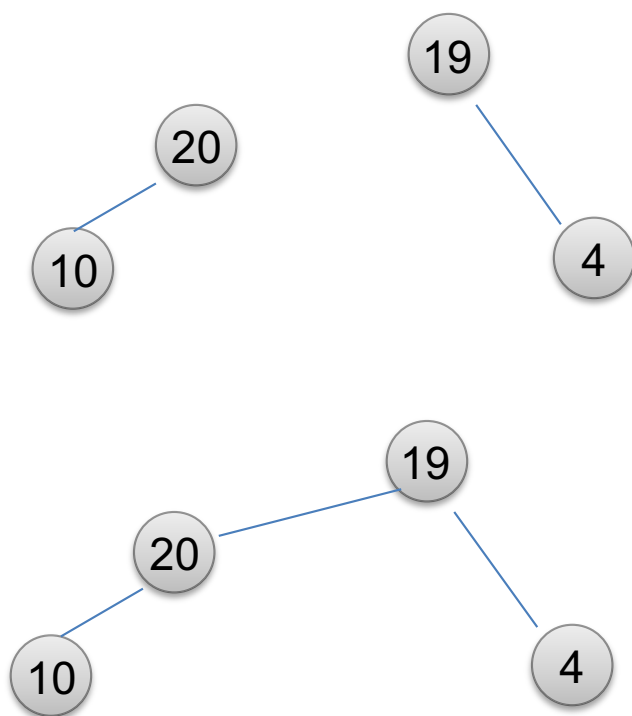


תרגיל 2

נתון גרף לא מכוון בעל n צמתים. בתחילה הגרף ריק (אין קשתות בגרף).
יש לתאר מבנה נתונים התומך בפעולות הבאות בסיבוכיות זמן טובה ככל האפשר:

Connect(x,y) חבר בין הצמתים x ו- y .

IsReachable(x,y) עונה על השאלה: האם ניתן להגיע מצומת x לצומת y בגרף.



`isReachable(19,20) = false`

`Connect(19,20)`

`isReachable(19,20) = true`

תרגיל 2

נתון גרף לא מכוון בעל n צמתים. בתחילה הגרף ריק (אין קשתות בגרף). יש לתאר מבנה נתונים התומך בפעולות הבאות בסיבוכיות זמן טובה ככל האפשר:

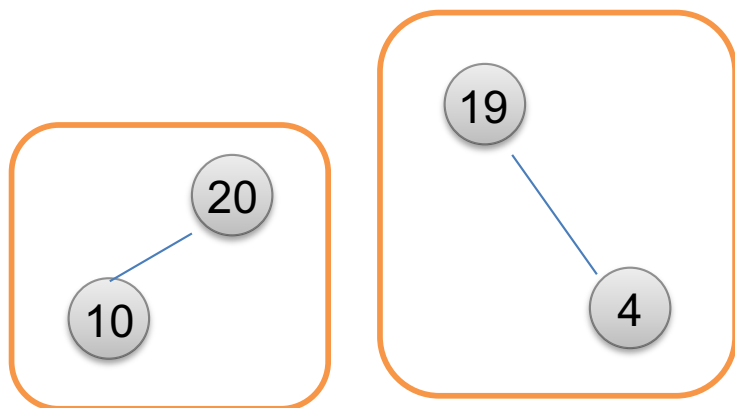
Connect(x, y) חבר בין הצמתים x ו- y .

IsReachable(x, y) עונה על השאלה: האם ניתן להגיע מצומת x לצומת y בגרף.

הקבוצות שלנו יהיו רכיבי הקשירות השונים.

במבחן כאשר פותרים עם Union-Find
צריך להגדיר –

1. מי האיברים.
2. מי הקבוצות הזרות.
3. מה השיטה.



`isReachable(19,20) = false`

`Connect(19,20)`

`isReachable(19,20) = true`

תרגיל 2

נתון גרף לא מכוון בעל n צמתים. בתחילה הגרף ריק (אין קשתות בגרף). יש לתאר מבנה נתונים התומך בפעולות הבאות בסיבוכיות זמן טובה ככל האפשר:

Connect(x, y) חבר בין הצמתים x ו- y .

IsReachable(x, y) עונה על השאלה: האם ניתן להגיע מצומת x לצומת y בגרף.

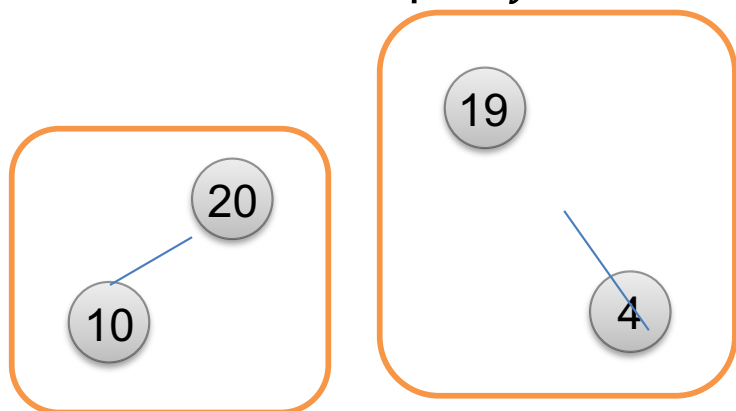
הקבוצות שלנו יהיו רכיבי הקשירות השונים.

במבחן כאשר פותרים עם Union-Find

צריך להגדיר –

1. מי האיברים.

2. מי הקבוצות הזרות.



פתרון

נחזיק מבנה נתונים Union Find. הקבוצות במבנה יהיו אוסף רכיבי הקשירות בגרף. כלומר, לכל איבר x , הקבוצה שבה הוא ימצא תכיל את כל האיברים שניתן להגיע מ- x אליהם.

Connect(x, y): $\text{if (Find}(x) \neq \text{Find}(y)) \quad \text{Union}(\text{Find}(x), \text{Find}(y))$
למה צריך find בפנים? בדיקה אם זה אותו רכיב קשירות

IsReachable(x, y): $\text{return (Find}(x) == \text{Find}(y))$

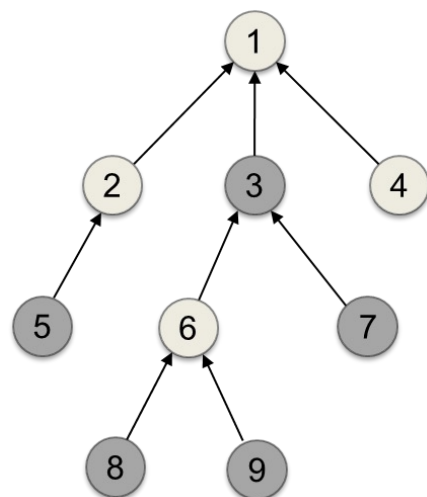
ניתוח סיבוכיות: שתי הפעולות Connect ו-IsReachable מבצעות כל אחת מספר קבוע של פעולות Union ו-Find. לכן, סיבוכיות הזמן המשוערכת שלהן היא $O(\log^* n)$.

תרגיל 2

ניתוח סיבוכיות

- אנחנו יודעים ש `isReachable` מבצעות מספר קבוע כלשהו של פעולות `union, find`.
- ניתן להגיד שסה"כ יש מספר קבוע של פעולות (שמורכב משני קבועים) של פעולות `union, find`.
- ב `isReachable` אנחנו מבצעים 2 פעולות `find`.
- אם אנחנו עושים m פעולות של `connect` and `isReachable` מה הוא סדר הגודל של כמות הפעולות של `union, find` שאנחנו מבצעים?
- כל פעולת `connect` קוראת לאיזשהו קבוע c כפול מספר הפעולות של `union, find`.
- כל פעולת `isReachable` כנ"ל קוראת לאיזשהו קבוע c כפול מספר הפעולות של `union, find`.
- סדר הגודל הוא $O(m)$.
- לכן לפי מה שלמדנו בהרצאה – סיבוכיות הזמן של סדרת m פעולות של `union, find` אנו יודעים מההרצאות שזה $O(m \log^* n)$ לכן כל אחת מהפעולות לבד זה $O(\log^* n)$ משוערך.
- הפעולות תלויות כי ייתכן ש `find` מסוים יהיה קשור ל `union` בכיווץ מסלולים וכו'.

תרגיל 3



נתון אילן יוחסין (קבוע מראש) המיוצג כעץ הפוך מכוון.

בתחילת הריצה כל הצמתים מסומנים כפעילים.

יש לממש את הפעולות הבאות ביעילות:

Close(x) בהינתן מצביע לצומת x בעץ, יש לסמן אותו כ"כבוי".

Master(x) מחזיר את האב הקדמון הקרוב ביותר של x באילן היוחסין

שמסומן כ"פעיל". אם x פעיל, אז $\text{Master}(x) = x$.

הנחה: אין לכבות את שורש אילן היוחסין.

ניתן לגשת לכל איבר באילו יוחסין ב $O(1)$.

מי המסטר של 4? 4 כי הוא פעיל

מי המסטר של 7? 1 (7 לא פעיל, 3 פעיל)

מי המסטר של 5? 2. ואם נעשה close 2 אז זה יהיה 1.

שימו לב: אנחנו עשויים לקבל גם אילן יוחסין כזה:



תרגיל 3

פתרון

נשתמש במבנה הנתונים Union Find. נחזיק קבוצה לכל צומת פעיל לפי הכלל הבא:

- כל צומת פעיל x יכיל בקבוצתו את אוסף הצמתים z עבורם $\text{Master}(z) = x$.
נחזיק ברשומה של כל קבוצה את שם הצומת הפעיל היחיד בקבוצה.

האיברים – הם הצמתים.

הקבוצות הזרות – כל הצמתים שחולקים את אותו מסטר פעיל.

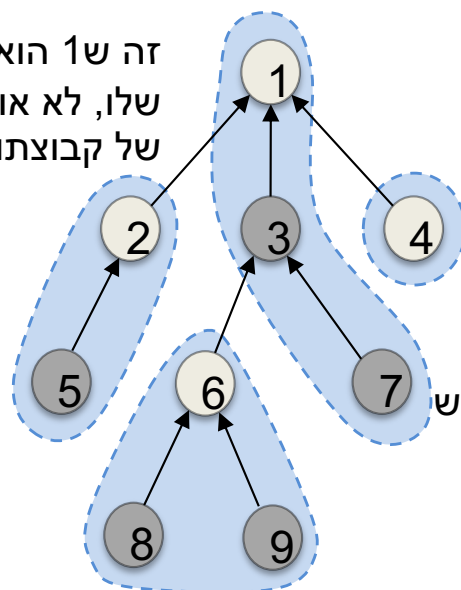
אתחול:

$\text{Master}(x)$: נגש לשורש הקבוצה בה נמצא x ונחזיר את שם הצומת הפעיל היחיד בקבוצה.

$\text{Close}(x)$: נסמן ב- y את אביו של x באילן היוחסין (נגיע אליו דרך אילן היוחסין הנתון לנו).

נבצע: $\text{Union}(\text{Find}(x), \text{Find}(y))$.

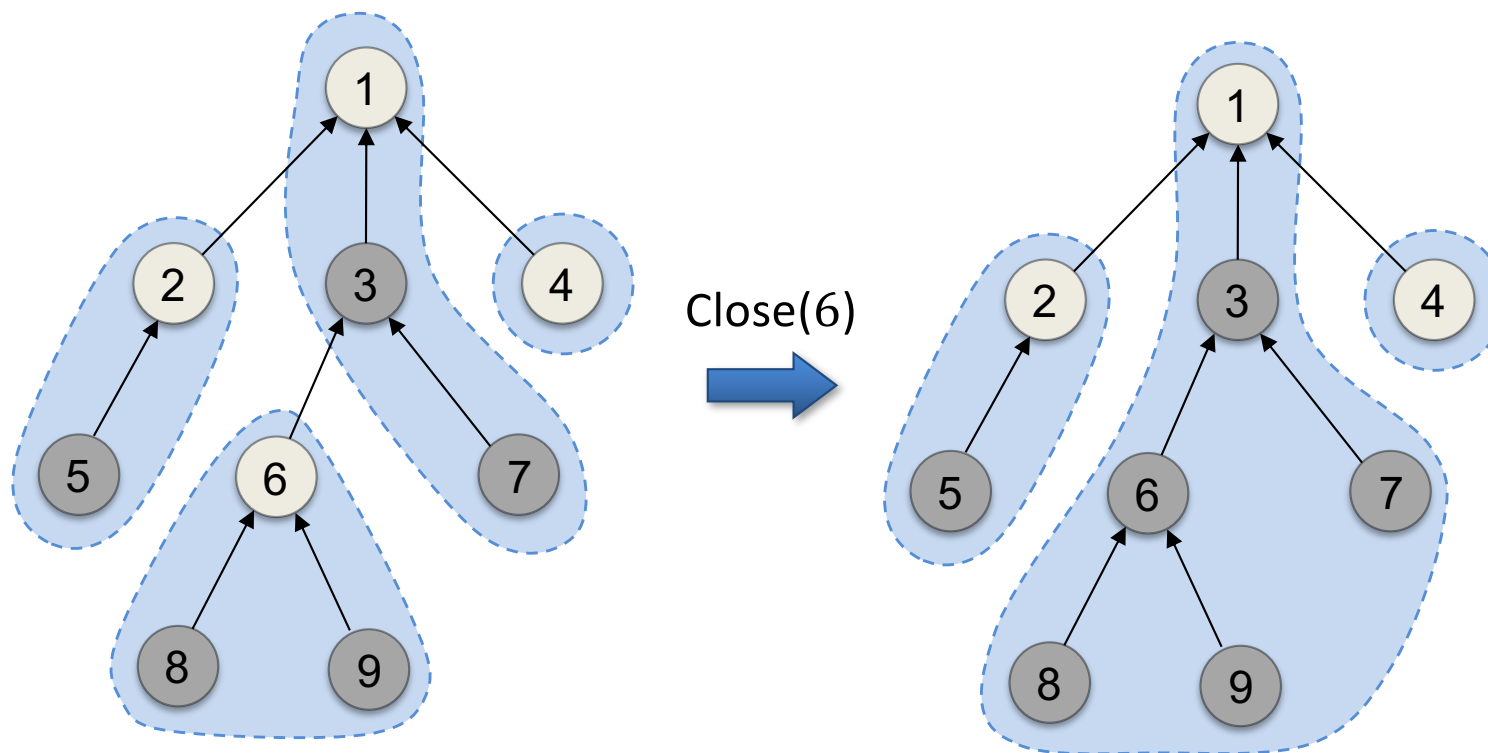
זה ש 1 הוא master בקבוצה שלו, לא אומר שהוא השורש של קבוצתו במבנה ה-uf.



הפעם נשמור בשדה מי הוא המסטר ולא מי הוא המקסימום.

יש מצב ששורש הקבוצה UF הוא בכלל 3.

תרגיל 3



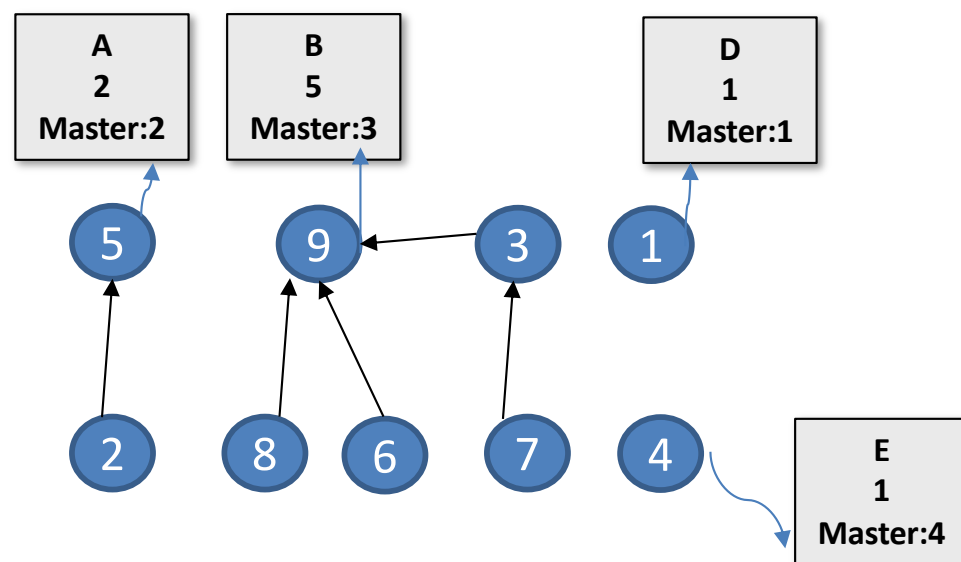
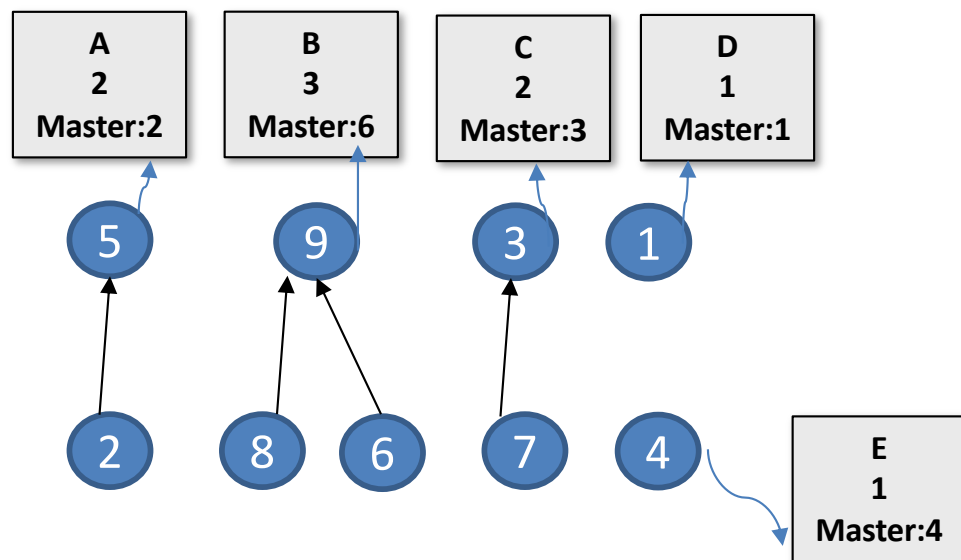
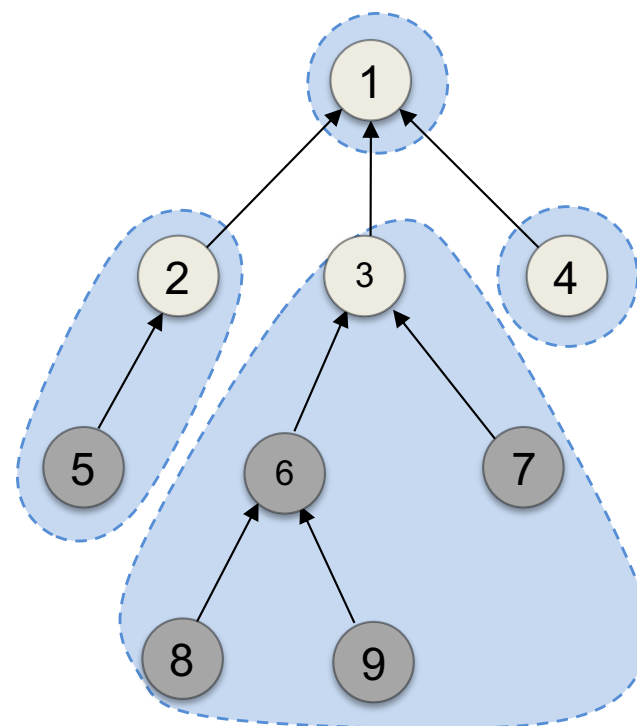
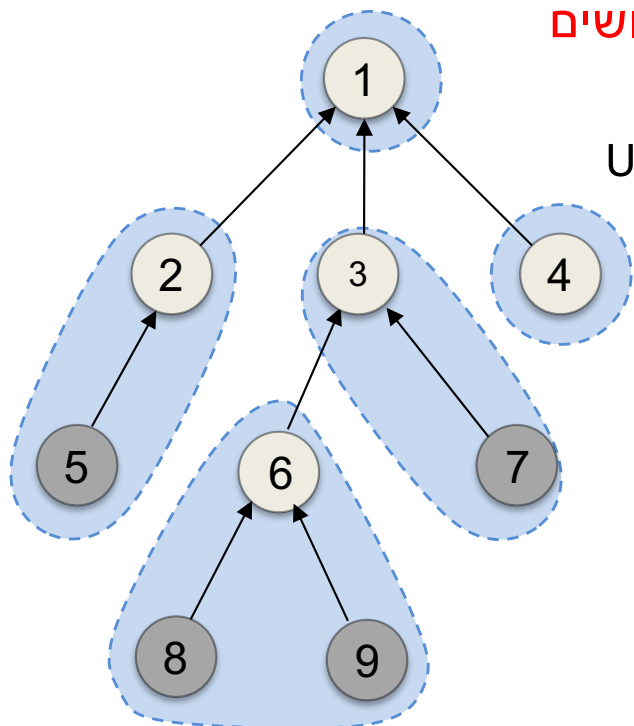
שימו לב: הציורים לעיל מתארים את אילן היוחסין ואת הקבוצות הזרות.
אלה הם לא העצים הפוכים במימוש של Union Find.

תרגיל 3

מוצאים את 6 ומוצאים את
האבא של 6 ולהם עושים
בעצם איחוד.

Union(Find(6),Find(3))

Close(6)



תרגיל 3

סיבוכיות –

עבור m פעולות Master-Close מתבצעות $O(m)$ פעולות Union-Find. לכן, במימוש ע"י עצים הפוכים, סיבוכיות הזמן של m פעולות היא $O(m \log^* n)$ במקרה הגרוע. מכאן, הסיבוכיות המשוערכת של פעולות אלו היא $O(\log^* n)$.

- אפשר להשתמש כמבנה נתונים של master-close במבחן. קלט האתחול של המבנה חייב להיות אילן יוחסין כמו שמוגדר בשאלה.
- אילן יוחסין יכול להיות גם שרוך.
- ההנחה של תחילת השאלה אמרה שאסור לכבות את השורש של אילן היוחסין, לכן אם זה לא מסתדר עם השאלה, אפשר להגדיר שורש פיקטיבי שתמיד דלוק.

תזכורת ורמז לתרגילים: אילן יוחסין יכול להיות גם שרוך (ולא בהכרח עץ הפוך מאוד מורכב):



טריק: אמרנו כהנחה שאסור לכבות את שורשו של אילן היוחסין. מה אם זה לא מסתדר עם השאלה? נגדיר שורש פיקטיבי!



תרגיל 4 – לא בטוח שנספיק - ארוך

נתונים n ארגזים. ארגז מוגדר ע"י: מזהה, אורך, רוחב, גובה.

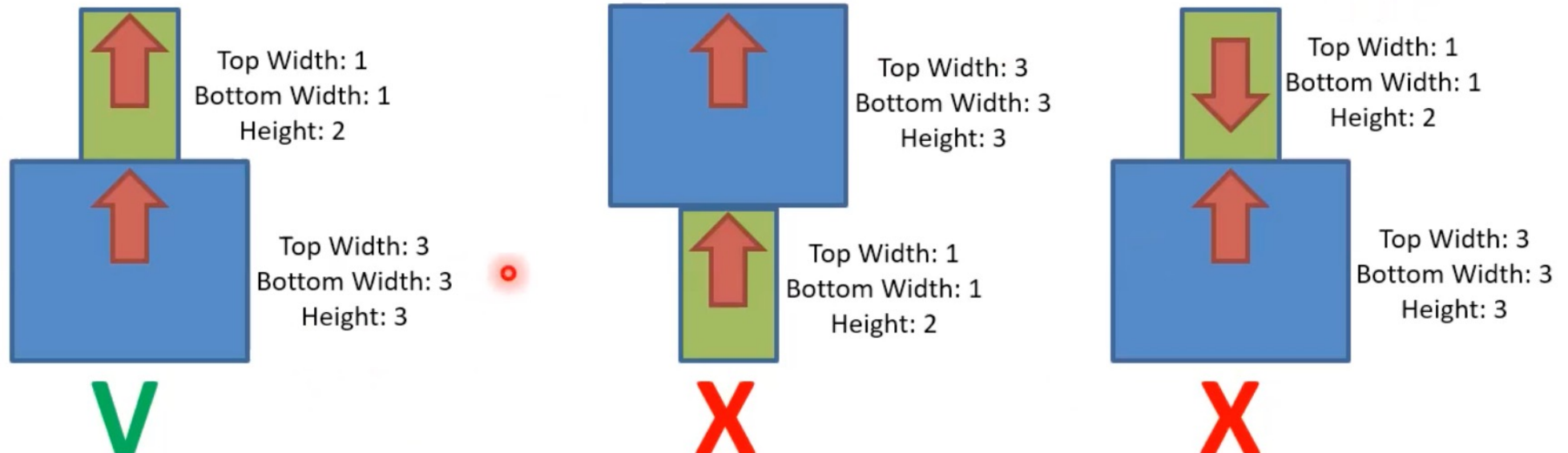
נרצה לבנות מארגזים אלו מגדלים.

חוקי בניית המגדלים:

1. לכל ארגז קיימת פאה אחת בדיוק עליה כתוב "This Side Up", אשר חייבת להיות מופנה כלפי מעלה.

2. ארגז יכול להיות מונח על ארגז אחר רק אם הפאה התחתונה שלו מוכלת ממש בפאה העליונה של הארגז עליו הוא מונח.

דוגמאות :



תרגיל 4 –

נתונים n ארגזים. ארגז מוגדר ע"י: מזהה, אורך, רוחב, גובה.

נרצה לבנות מארגזים אלו מגדלים.

דרוש מבנה נתונים התומך בפעולות הבאות:

Init($Boxes, n$) מאתחלת מחסן שיכיל את n הארגזים הנתונים בקלט $Boxes$.

הארגזים מזהים ע"י המספרים $1, \dots, n$ וכל ארגז מהווה מגדל.

PutOnTop($B1, B2$) מניחה את המגדל בו נמצא הארגז $B1$ על המגדל בו נמצא הארגז $B2$.

(בתנאי שניתן לעשות זאת בלי להפר את חוקי הבניה).

TowerHeight(B) מחזירה את גובה המגדל בו נמצא הארגז B .

סיבוכיות הזמן של פעולת האתחול הינה $O(n)$.

סיבוכיות הזמן המשוערכת של שתי הפעולות האחרונות ביחד הינה $O(\log^* n)$.

תרגיל 4

נשתמש ב-UF, כאשר כל קבוצה מייצגת מגדל, וכל איבר מייצג ארגז (בהתחלה כל ארגז הוא מגדל).

כל קבוצה תשמור את המידע הנוסף הבא:

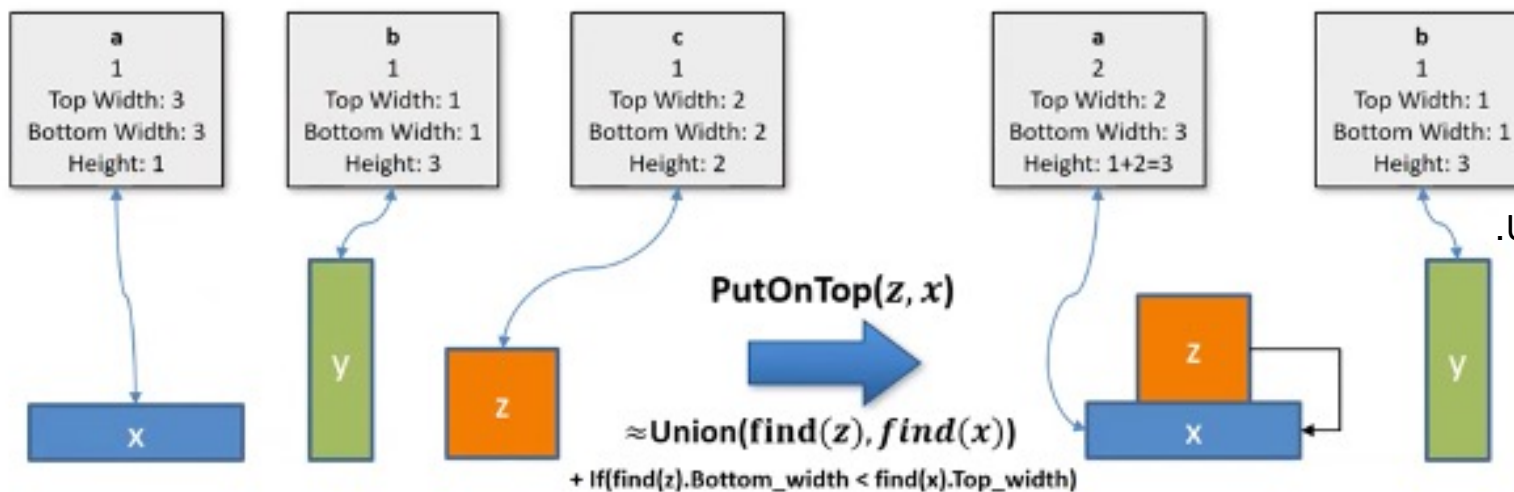
- גודל הפאה של הארגז התחתון במגדל - בשביל שנדע אילו מגדלים אפשר לשים מתחתיו.
- גודל הפאה של הארגז העליון במגדל – בשביל שנדע אילו מגדלים אפשר לשים מעליו.
- גובה המגדל.

תרגיל 4

נשתמש ב-UF, כאשר כל קבוצה מייצגת מגדל, וכל איבר מייצג ארגז

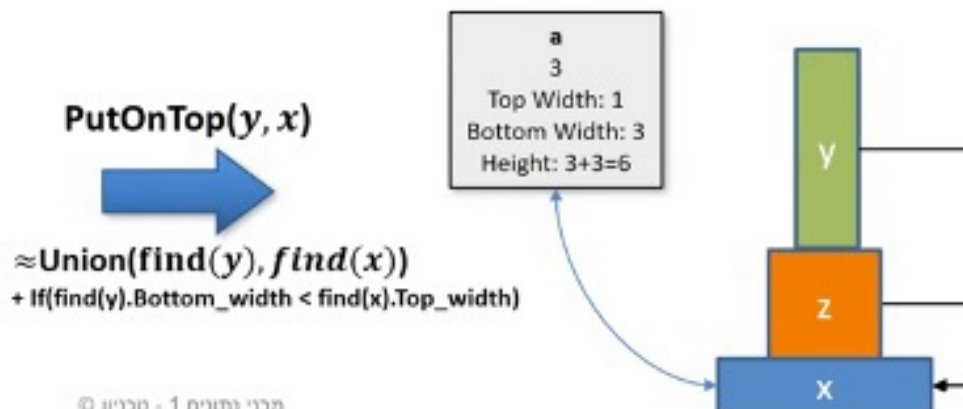
כל קבוצה תשמור את המידע הנוסף הבא:

- גודל הפאה של הארגז התחתון במגדל
- גודל הפאה של הארגז העליון במגדל
- גובה המגדל



- שורש הקבוצה קשור רק לUF.
- הארגזים הם הצמתים.

צריך לוודא את תנאי הפאות בשביל לקיים את הנכונות של השאלה.



- כאשר נרצה לחפש $TowerHeight(z) = find(z).height..$

תרגיל 4

– **PutOnTop(B_1, B_2)**

- נבצע $T_1 \leftarrow \text{find}(B_1), T_2 \leftarrow \text{find}(B_2)$
- נוודא כי גודל הפאה התחתונה ב- T_1 מוכלת בגודל הפאה העליונה של T_2 .
- אם כן, נבצע $\text{Union}(T_1, T_2)$ ונעדכן את המידע הנוסף בקבוצה החדשה (ניתן לחישוב ב- $O(1)$)

TowerHeight(B) – נבצע $\text{find}(B)$ ונחזיר את גובה המגדל השמור במידע הנוסף של הקבוצה

בכל אחת משתי הפעולות האחרונות נבצע מספר קבוע של פעולות union ו- find , ולכן סיבוכיות הזמן המשוערכת של שתיהן היא $O(\log^* n)$

נרצה להוסיף למימוש את הפעולה הבאה:

HeightFromGround(B) מחזירה את גובה הפאה התחתונה של הארגז B מהרצפה.

סיבוכיות הזמן המשוערכת של שלוש הפעולות האחרונות ביחד הינה $O(\log^* n)$.

תרגיל 4

נרצה להוסיף למימוש את הפעולה הבאה:

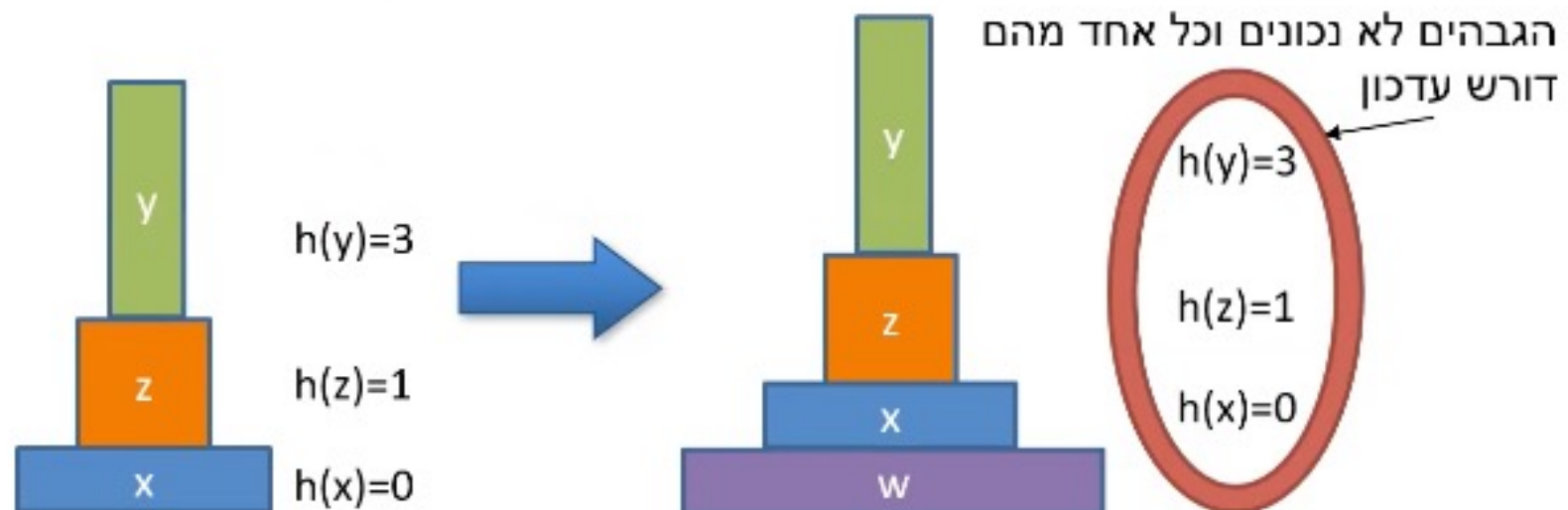
HeightFromGround(B) מחזירה את גובה הפאה התחתונה של הארגז B מהרצפה.

סיבוכיות הזמן המשוערכת של שלוש הפעולות האחרונות ביחד הינה $O(\log^* n)$.

למה זה קשה?

זה לא יהיה $\log^* n$

הבעיה: אם תחזקנו את הפעולה באמצעות מידע נוסף בכל צומת ששומר את גובה הארגז מהקרע, ישנן פעולות שידרשו עדכון ככמות האיברים בקבוצה של המגדל שהוסף מלמעלה:

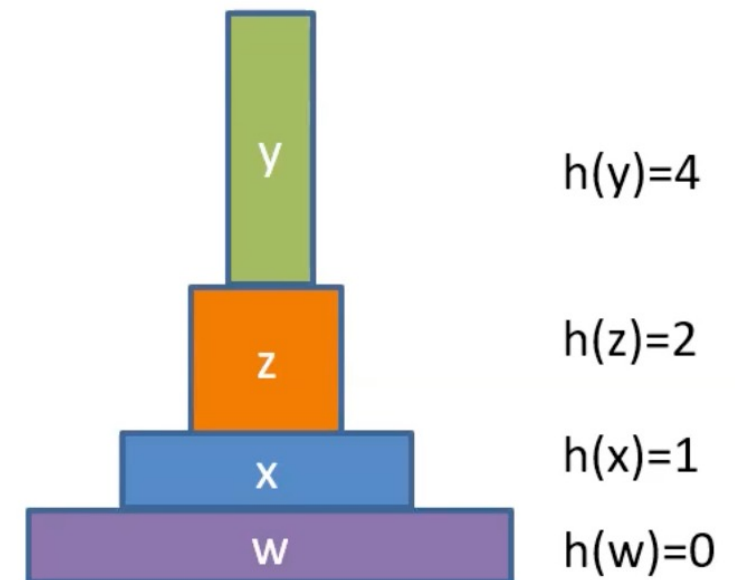
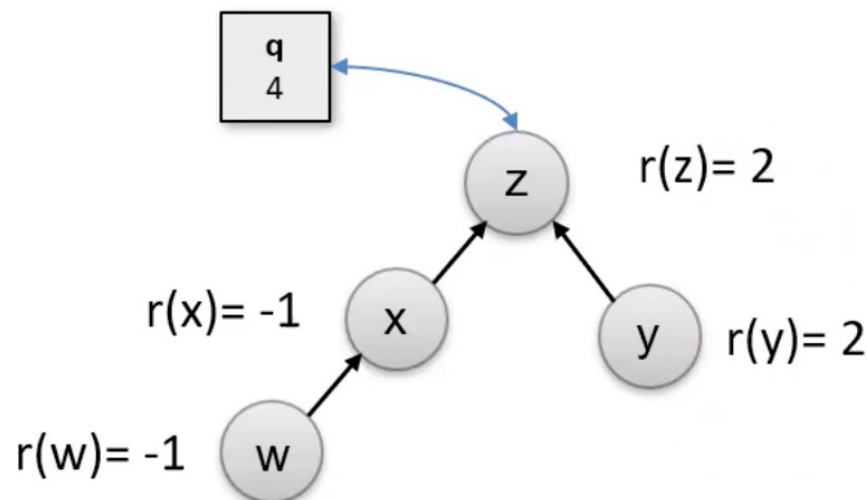


לעדכן את כל אחד מהשדות יעלה במקרה הגרוע $O(n)$!

תרגיל 4

אינטואיציה לדרך הפתרון:

היינו רוצים להגדיר שדה נוסף לכל צומת במבנה כך שכשנעשה את פעולת **HeightFromGround** באמצעות **Find**, סכימת השדות לאורך מסלול ה-**Find** תהיה שווה לגובה הארגז מהקרקע.



חיבור ה- r ייתן לנו את הגובה האמיתי של הארגז.

$h(x)$ * הוא הגובה האמיתי של ארגז x מהקרקע.

$r(x)$ ** הוא ערך השדה הנוסף שאנו שומרים עבור צומת x במבנה.

עכשיו רק נותר להראות כיצד נעשה זאת טכנית.

$$h(z)=r(z)=2$$

$$h(w)=r(w)+r(x)+r(z)=0$$

$$h(x)=r(x)+r(z)=1$$

$$h(y)=r(y)+r(z)=4$$

תרגיל 4

נראה שאנחנו מסוגלים לתחזק את השדה r לאורך חיוו של המבנה כך שסכימתו לאורך מסלול Find מכל צומת (ארגז) ועד לשורש הקבוצה תביא לגובה מהקרקע תקין עבור הצומת.

אתחול: בהתחלה, גובהם של כל הארגזים מהקרקע הוא 0. לכן נאתחל לכל צומת במבנה $r(x)=0$.

תרגיל 4

נכונות התכונה תו"כ פעולות המבנה: נרצה להראות שנוכל לתחזק את הערך r גם עם פעולות **PutOnTop**. נניח שלפני הפעולה הנוכחית, התכונה התקיימה עבור שני מגדלי ארגזים A ו-B. תמיד נרצה לשים את B על A מבחינת המגדלים, אבל מבחינת הקבוצות במבנה ה-UF יש לנו שתי אפשרויות:

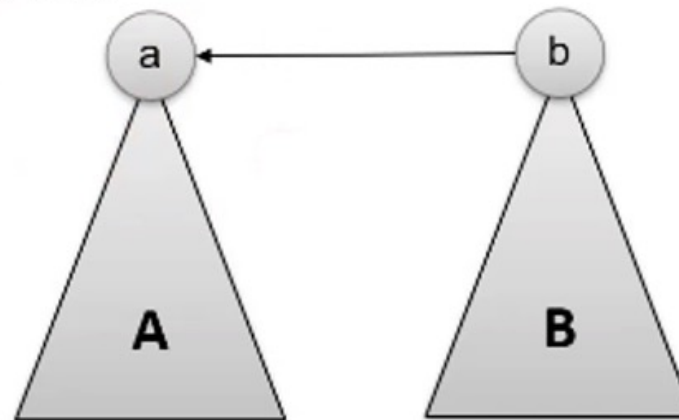
אפשרות א':
 $|A| \geq |B|$

העצים הפוכים של UF

מגדלי הארגזים



$$r(a)_{new} = r(a)_{old}$$



האם הערכים A-B השתנו עקב הפעולה?
לא, הארגזים A-B שמרו על אותו מרחק יחסי מהקרקע.

תרגיל 4

נכונות התכונה תו"כ פעולות המבנה: נרצה להראות שנוכל לתחזק את הערך r גם עם פעולות **PutOnTop**. נניח שלפני הפעולה הנוכחית, התכונה התקיימה עבור שני מגדלי ארגזים A ו-B. תמיד נרצה לשים את B על A מבחינת המגדלים, אבל מבחינת הקבוצות במבנה ה-UF יש לנו שתי אפשרויות:

אפשרות א':
 $|A| \geq |B|$

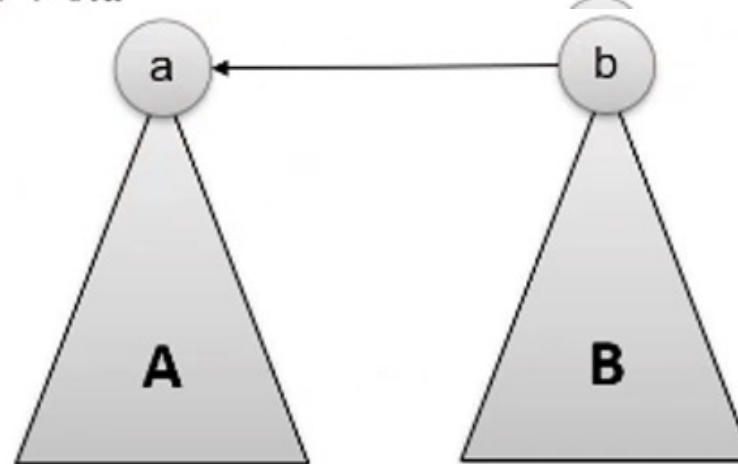
העצים ההפוכים של UF

מגדלי הארגזים



$$r(a)_{new} = r(a)_{old}$$

$$r(b)_{new} = r(b)_{old} + h(A) - r(a)_{new}$$



האם הערכים ב-B השתנו עקב הפעולה? כן, הארגזים ב-B גבהו ב- $h(A)$ מהקרקע.

$$\text{HeightFromGround}(b) = r(b)_{old} + \underbrace{h(A) + r(a)_{new}}_{\text{ספרנו כאן את } r(a)_{new} \text{ פעמיים!}}$$

צריך לתקן את זה ב- $r(b)_{new}$

$$r(b)_{new} = r(b)_{old} + h(A) - r(a)_{new}$$

תרגיל 4

אפשרות ב':

$$|A| < |B|$$

מגדלי הארגזים

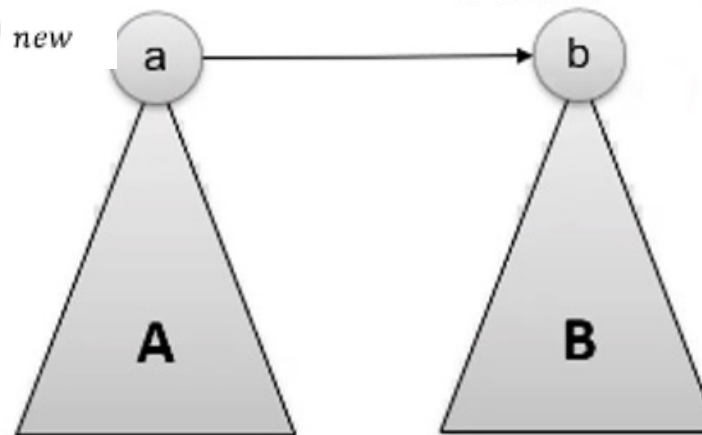
העצים ההפוכים של UF



$$r(a)_{new} = r(a)_{old} - r(b)_{new}$$

**האם הערכים ב-A
השתנו עקב
הפעולה?**

הארגזים ב-A שמרו על
אותו מרחק יחסי
מהקרקע, אבל מסלול
החיפוש של Find עכשיו
עובר דרך b, לכן נצטרך
לתקן את $r(a)_{new}$.



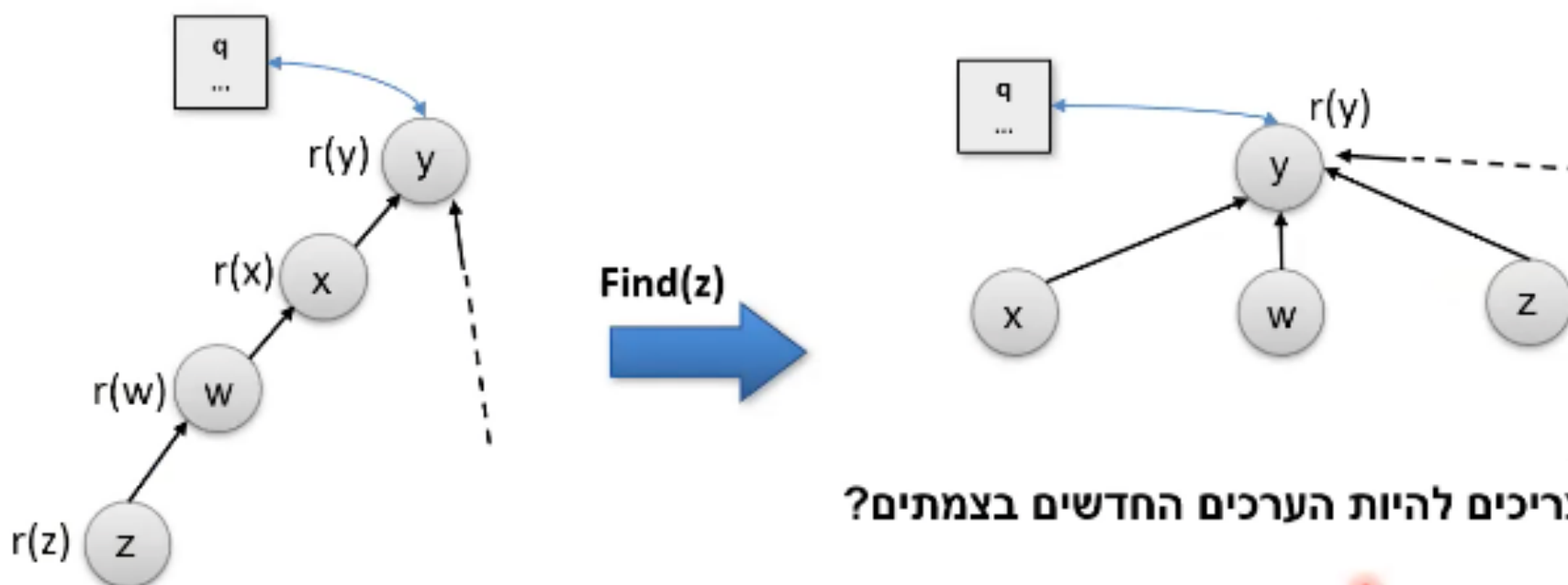
$$r(b)_{new} = r(b)_{old} + h(A)$$

**האם הערכים ב-B
השתנו עקב
הפעולה?**

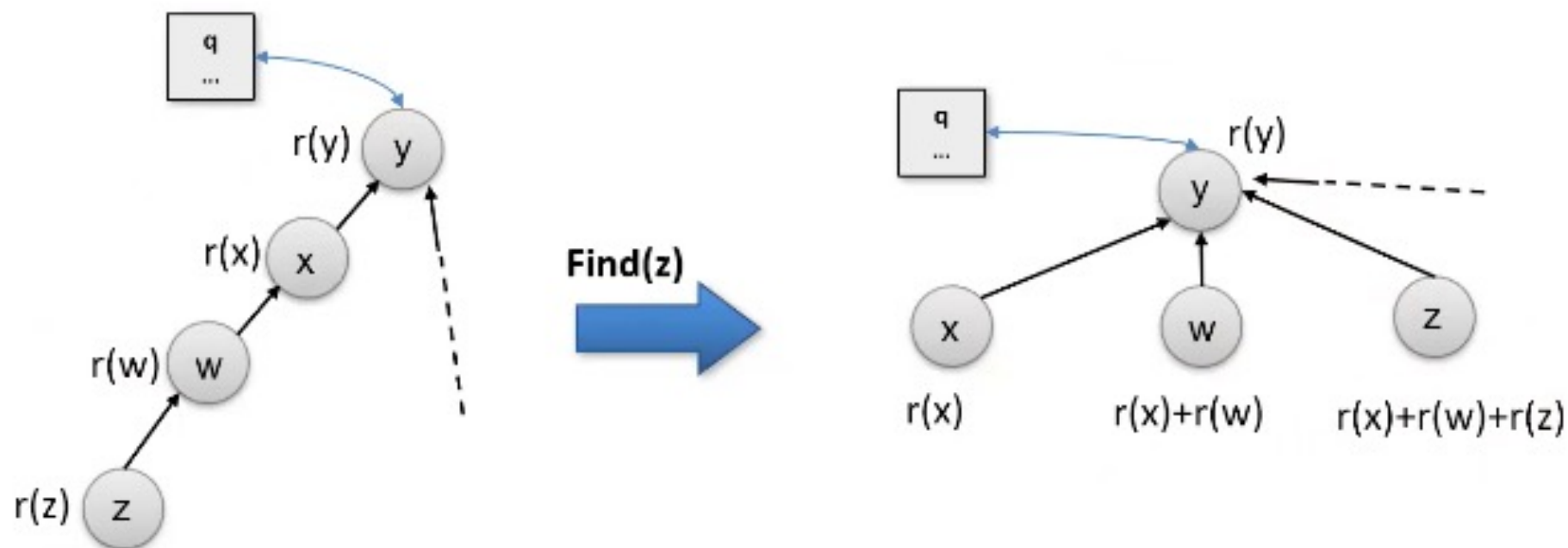
כן, הארגזים ב-B גבהו
ב- $h(A)$ מהקרקע.
הפעם מסלול ה-Find לא
יעבור ב-a ולכן אין צורך
לתקן את $r(b)_{new}$.

תרגיל 4

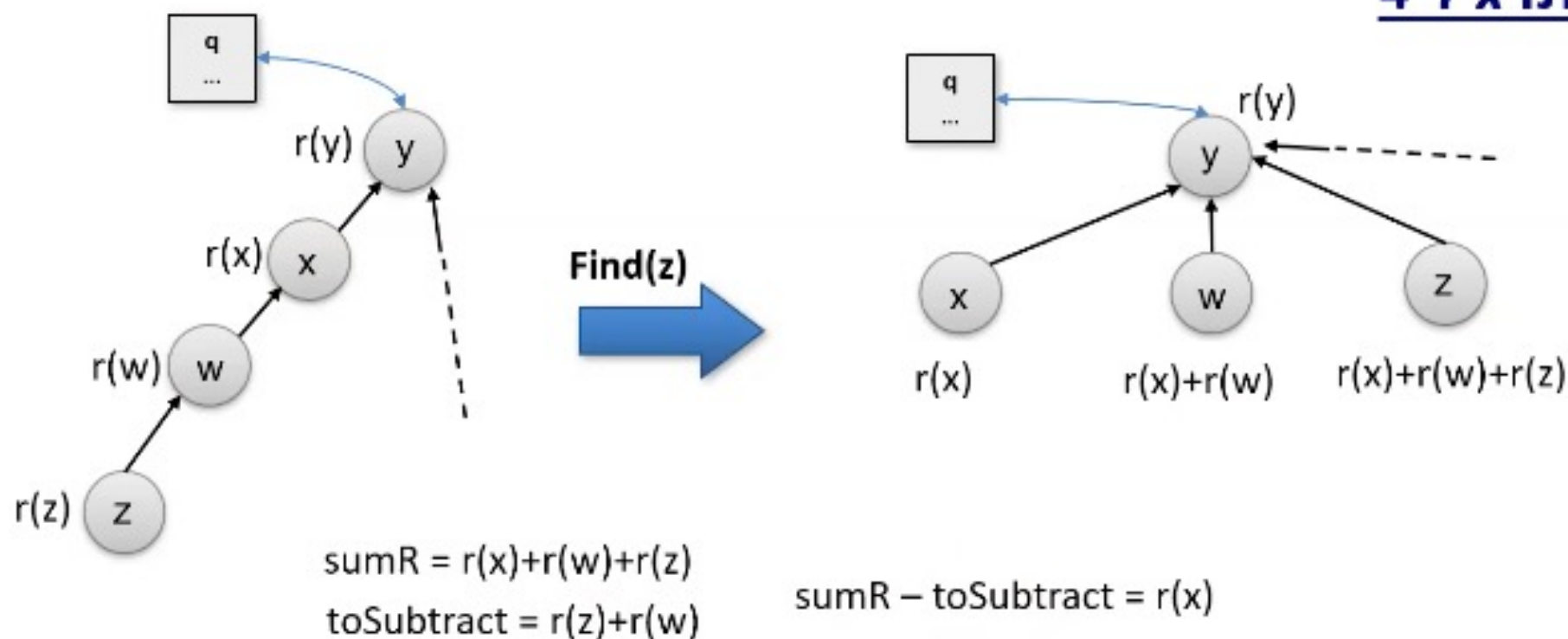
נכונות התכונה תו"כ פעולות המבנה: נרצה להראות שנוכל לתחזק את הערך r גם אם נכוץ מסלולים בביצוע פעולות Find (נצטרך לקרוא לה בביצוע כל אחת מהפעולות שהגדרנו):



תרגיל 4



תרגיל 4



דרך חישוב:

- 1) במעבר הראשון של Find (בו רק מחפשים את השורש לקראת כיוון המסלולים) נסכום את הערכים עד לשורש (לא כולל).
- 2) במעבר השני נחזיק סכום של הערכים במסלול החיפוש עד לצומת הנוכחי (לא כולל), ונבצע החסרה מהסכום הכולל בשביל ערך השדה החדש של הצומת.